ORIENTATION TO COMPUTING

UNIT 1: FUNDAMENTALS OF COMPUTING, COMPUTER ORGANIZATION & ARCHITECTURE
Curriculum: Lovely Professional University | School of Computer Science & Engineering
Reference: Data Communications and Networking by Behrouz A. Forouzan & Computer Organization by Carl Hamacher

>> 1.1 Historical Evolution and Generations of Computing

A computer is a programmable electronic device that accepts raw data, processes it according to stored instructions, and produces meaningful information:
* 1st Generation (1940-1956): Vacuum Tubes. Magnetic drums for memory; programmed in raw Machine Language; massive heat dissipation (e.g., ENIAC, UNIVAC).
* 2nd Generation (1956-1963): Transistors. Invented at Bell Labs; smaller, faster, energy-efficient; magnetic core memory; introduction of Assembly & early high-level languages (FORTRAN, COBOL).
* 3rd Generation (1964-1971): Integrated Circuits (ICs). Semiconductor chips packing dozens of transistors; keyboards and monitors replaced punched cards; Operating Systems emerged (IBM System/360).
* 4th Generation (1971-Present): Very Large Scale Integration (VLSI) & Ultra Large Scale Integration (ULSI). Complete CPU integrated onto a single microchip (Intel 4004 in 1971); personal computers (PC), laptops, smartphones.
* 5th Generation (Present and Beyond): Artificial Intelligence, quantum computing, parallel processing, and neuromorphic computing architectures.

>> 1.2 Functional Hardware Subsystems of a Digital Computer

Every digital computer is composed of five interconnected functional units:
* 1. Input Unit: Transduces human-readable or physical real-world data into binary electrical signals (Keyboards, mouse, optical scanners, sensors, ADCs).
* 2. Central Processing Unit (CPU) - The Brain:
- Arithmetic Logic Unit (ALU): Executes arithmetic operations (addition, subtraction, multiplication) and logical operations (AND, OR, NOT, comparisons).
- Control Unit (CU): The supervisor of the CPU. Fetches instructions sequentially from memory, decodes opcodes, and emits precision micro-electrical timing and control signals.
- Register Array: Ultra-high-speed internal CPU storage locations operating at processor clock speed.
* 3. Memory Subsystem:

- Primary Memory: Directly accessible by CPU. RAM (Random Access Memory - volatile read/write) and ROM (Read Only Memory - non-volatile firmware/BIOS/UEFI).
- Cache Memory: Static RAM (SRAM) operating near CPU core frequency to bridge the processor-memory speed gap (L1 Cache: fastest, private to core; L2 Cache; L3 Cache: large, shared across cores).
- Secondary Memory: Magnetic hard disk drives (HDD), Solid State Drives (NVMe/SATA SSD), optical discs; non-volatile persistent mass storage.

* 4. Output Unit: Translates processed binary data back into human-perceptible form (Displays, printers, audio synthesizers, motor actuators).

>> 1.3 Computer Architecture Models: Von Neumann vs Harvard

* Von Neumann Architecture (1945):
- Core Paradigm: Stored-Program Concept. Both machine instructions (code) and program data share a single unified memory space and unified system bus.
- The Von Neumann Bottleneck: Because code and data share the same physical bus, the CPU cannot fetch an instruction and read/write operand data simultaneously. Throughput is strictly bounded by memory bus throughput.

* Harvard Architecture:
- Core Paradigm: Physically separated memory banks and dedicated independent buses for instructions and data.
- Advantage: Simultaneous instruction fetching and operand memory access, yielding higher execution throughput. Widely implemented in Digital Signal Processors (DSPs) and modern CPU L1 caches (split L1-Instruction and L1-Data caches).

>> 1.4 Internal CPU Registers and Machine Instruction Cycle

* Dedicated CPU Registers:
- Program Counter (PC): Holds the memory address of the next instruction to be fetched.
- Memory Address Register (MAR): Holds the physical memory address currently being read from or written to.
- Memory Data Register (MDR) / Memory Buffer Register (MBR): Holds the actual data/instruction read from or written to memory.
- Instruction Register (IR): Holds the fetched instruction word while it is being decoded by the Control Unit.
- Accumulator (ACC) & General Purpose Registers (R0-R15): Hold intermediate mathematical operands and computation results.

* The 4-Phase Instruction Execution Cycle:
- Phase 1: Fetch (PC -> MAR -> Address Bus -> Memory -> Data Bus -> MDR -> IR; PC incremented).

- Phase 2: Decode (Control Unit decodes opcode in IR, determines required operands and addressing modes).
- Phase 3: Execute (ALU performs specified mathematical or logical operation on operands).
- Phase 4: Store / Writeback (Result stored back into destination register or written to memory via MDR/MAR).

>> 1.5 System Bus Interconnections and Bandwidth Theory

* Data Bus: Bidirectional bus carrying raw data values between CPU, memory, and peripherals. Width determines word size (32-bit, 64-bit).
* Address Bus: Unidirectional bus driven by CPU carrying physical memory addresses. Width N determines maximum addressable memory capacity = 2^N bytes.
* Control Bus: Carrying synchronization signals (Memory Read, Memory Write, I/O Read, I/O Write, Bus Request, Bus Grant, Interrupt Request, Clock).
* Bus Bandwidth Formula: Bandwidth (Bytes/sec) = Bus Width (Bytes) x Bus Frequency (Hz) x Transfers per Clock.

[PROBLEM 1: Register-Level Trace of Instruction Fetch-Execute Cycle]

Problem Statement: Trace the step-by-step register states during the execution of an assembly instruction "ADD [0x0500]" (Opcode: Add content of memory location 0x0500 to Accumulator). Initial conditions: PC = 0x0200, Accumulator (ACC) = 0x002A (Decimal 42). Memory location 0x0200 holds instruction opcode 0xA10500. Memory location 0x0500 holds data 0x0015 (Decimal 21).

Step-by-Step Register Transfer Language (RTL) Trace:

* 1. Fetch Cycle Step 1: MAR <- PC (MAR loaded with address 0x0200).
* 2. Fetch Cycle Step 2: Control Unit emits Memory Read. MDR <- Memory[MAR] (MDR receives 0xA10500). PC <- PC + 1 (PC becomes 0x0201).
* 3. Fetch Cycle Step 3: IR <- MDR (Instruction Register loaded with 0xA10500).
* 4. Decode Cycle: Control Unit decodes IR: Opcode = ADD, Operand Address = 0x0500.
* 5. Execute Cycle Step 1 (Operand Fetch): MAR <- 0x0500; Control Unit emits Memory Read; MDR <- Memory[0x0500] (MDR receives 0x0015).
* 6. Execute Cycle Step 2 (ALU Operation): ALU adds ACC (0x002A) + MDR (0x0015) = 0x003F (Decimal 63).
* 7. Writeback Cycle: ACC <- 0x003F. Status flags (Zero, Sign, Overflow) updated. Cycle completes.

[PROBLEM 2: Memory Address Space and Bus Bandwidth Derivation]

Problem Statement: An engineer evaluates two microprocessor architectures: System A with a 32-bit address bus and 32-bit data bus; System B with a 64-bit address bus and 64-bit data bus operating with dual-channel DDR4 RAM clocked at 3200 MT/s (MegaTransfers/sec). Calculate: (1) Maximum physical RAM addressable by each system, and (2) Peak memory bandwidth of System B.

Mathematical Derivation:

* 1. Maximum Addressable RAM Capacity:
- System A (32-bit address bus): Addressable locations = 2^32 = 4,294,967,296 bytes = 4 GB (Gigabytes).
- System B (64-bit address bus): Addressable locations = 2^64 = 1.844 x 10^19 bytes = 16 Exabytes (EB).
* 2. Peak Memory Bandwidth Calculation for System B:
- Data bus width per channel = 64 bits = 8 Bytes.
- Dual-channel total data bus width = 2 x 8 Bytes = 16 Bytes.
- Transfer rate = 3200 x 10^6 transfers per second.
- Peak Bandwidth = 16 Bytes x 3200 x 10^6 transfers/sec = 51,200 MB/s = 51.2 GB/sec.

[PROBLEM 3: Multi-Level Cache Memory Performance Analysis]

Problem Statement: A modern computing processor features a two-level cache hierarchy: L1 Cache with access time of 1 nanosecond (ns) and 92% hit rate; L2 Cache with access time of 5 ns and 98% local hit rate; Main Memory (RAM) with access time of 50 ns. Calculate the Effective Memory Access Time (EMAT).

Mathematical Derivation:

* 1. Given Parameters: t_L1 = 1 ns, H1 = 0.92, Miss Rate 1 (M1) = 1 - 0.92 = 0.08.
* 2. L2 Parameters: t_L2 = 5 ns, Local Hit Rate H2 = 0.98, Local Miss Rate M2 = 1 - 0.98 = 0.02.
* 3. Main Memory: t_RAM = 50 ns.
* 4. EMAT Formulation:
- EMAT = t_L1 + M1 x (t_L2 + M2 x t_RAM)
- EMAT = 1 ns + 0.08 x (5 ns + 0.02 x 50 ns)
- EMAT = 1 ns + 0.08 x (5 ns + 1.0 ns) = 1 ns + 0.08 x 6.0 ns = 1 ns + 0.48 ns = 1.48 ns.
* 5. Engineering Conclusion: Despite main memory requiring 50 ns, the high-speed multi-level cache reduces average access delay to just 1.48 ns (a 33.7x performance speedup).

[PROBLEM 4: Architectural Case Study: Von Neumann vs Modified Harvard]

Problem Context: Analyze why modern x86-64 and ARM processors are classified as Modified Harvard architectures rather than pure Von Neumann systems.

In-Depth Architectural Analysis:

* 1. External Memory Level (Von Neumann Aspect): From the operating system and programmer perspective, main memory (DRAM) is a single, unified flat address space containing both executable binaries and user data. Both share external memory channels.
* 2. Internal Microarchitectural Level (Harvard Aspect): Inside the CPU silicon die, the L1 cache is strictly physically partitioned into two independent caches: L1-Instruction Cache (L1-I) and L1-Data Cache (L1-D), driven by separate internal buses.
* 3. Pipelining Benefits: The instruction fetch stage reads from L1-I simultaneously while the memory load/store execution stage reads/writes to L1-D in the exact same clock cycle without structural hazards or bus contention.
* 4. Hardware Realization: This hybrid design eliminates the Von Neumann bottleneck in the high-frequency pipeline while retaining the software programming simplicity of a single unified address space.